FOUNDATION_PACKAGES

Foundation Packages (`pkg/`)

This catalogue describes the pure-Go foundation library that the Helix control plane is built from. There are **129+ packages** under `pkg/` (plus 19 private subsystems under `internal/`).

Every foundation package follows the same engineering contract:

- **Pure Go, standard-library-first.** External dependencies are avoided; where an in-repo primitive is reused it is imported read-only.
- **Deterministic.** Clocks, randomness, and I/O sources are injected (interfaces / seeds / explicit ticks) so tests are reproducible. No `time.Now()` inside logic under test.
- **Mutation-proven (CLAUDE-1).** Each package's tests are designed to *fail* if the specific logic they cover is mutated — green tests over stubs or always-true conditions are forbidden. Items are not marked complete without an independent mutation bite confirming the guard is live.
- **Cross-platform parity (CLAUDE-2).** OS-specific capabilities use real native facilities per platform; there are no Linux-only mocks behind build tags for real-operation features.

Consensus & Coordination

Package	Purpose
`voting`	Largest-subcluster-wins quorum (3/2 survive+fence, 2/2 both-fence, TTL re-resolve).
`failconfirm`	Redis-cluster two-phase failure confirmation (PFAIL→FAIL via distinct-reporter majority quorum, OnDead-once).
`heartbeatcoalescer`	Multi-Raft per-peer heartbeat coalescing, O(shards·peers)→O(peers), liveness preserved.
`leader`, `lock`	Leadership and advisory locking primitives.
`splitbrain`, `splitbrainalert`	Split-brain detection and alerting.

Package	Purpose
`phasegate`	Sub-phase dependency-gate validation.
`epochresolve`	Config-epoch failover slot-ownership conflict resolution (highest-epoch wins, deterministic lexicographic equal-epoch tiebreak, order-independent convergence).
`multiraft`	`MultiRaftManager` partitioning cluster state into independent per-shard `go.etcd.io/raft/v3` groups (own leader each) with per-shard `Propose` routing and an in-process `RaftTransport`; write throughput scales with shard count instead of hitting the single-leader ceiling, and a shard re-elects on leader loss while others keep committing. Includes a `LeaseTracker` for CockroachDB-style leaseholder local reads (no Raft round-trip; follower reads route via `RaftTransport.SendRead`; lease expiry re-routes the fast path). Durability-correct Ready loop: a `ShardStorage` `SetHardState`/`Append` persistence error **parks** the un-persisted `Ready` and skips `Advance` so raft re-presents it (never falsely marking it durably handled) (HXC-917); the persist-error is also surfaced through a public observability seam — a registerable `OnPersistError` hook + `LastPersistError` accessor fire with the shard id + error, nil hook = unchanged behavior (HXC-927).
`kraft`	KRaft-style self-managed Raft metadata quorum (`go.etcd.io/raft/v3`, no ZooKeeper): replicated `CreateTopic`/`ListTopics` + partition-leader assignment via a controller quorum, with controller election and controller-loss failover.
`stonith`	Shoot-The-Other-Node-In-The-Head fencing: a `FencingAgent` interface with IPMI / AWS-EC2 / Azure-ARM / SBD-shared-disk / NoOp drivers and a `MultiLevelFencer` multi-level fallback, with sink-side fence confirmation (target reachable→unreachable).

Membership & Discovery

Package	Purpose
`swim`	SWIM gossip membership with phi-accrual failure detection and hierarchical tiers.
`discovery` (+ `discovery/federated`, + `mdns.go`)	Service discovery, including federated cross-cell discovery and zero-config mDNS/DNS-SD LAN discovery on `_helix-cluster._tcp` (stdlib `net` + `x/net/dnsmessage`): an advertiser publishing TXT records (cellid/nodeid/version/wgpubkey/clusteraddr) and a browser parsing entries into `DiscoveredPeer` while rejecting any missing/invalid required field. Discovery only — trust requires SPIFFE attestation afterward (live two-node LAN capture tracked as HXC-925).
`nattraversal`	STUN-based NAT traversal.
`ice`	ICE connectivity establishment.
`cellmesh`	Cross-cell mesh networking.
`scan`	Oracle-SCAN-style stable virtual client endpoint: least-loaded healthy-backend routing with a membership-stable name and injected clock.

Replication & State

Package	Purpose
`crdt` (+ `crdt/merkle`)	LWW / OR-Set / G-Counter / PN-Counter / vector-clock CRDTs and Merkle anti-entropy trees.
`mvcc`	MVCC revision store with a real B-tree index, time-travel reads, watch-from-revision, and compaction.
`antientropy`	Cassandra-style 3-layer repair: hinted handoff, read repair, Merkle-tree diff (over in-memory replicas).
`watchmanager`	etcd-style persistent watch manager with synced / unsynced / victim watcher groups.
`hlc`	Hybrid logical clocks.
`offlinesync`, `checkpoint_merge`	Offline delta sync and checkpoint merging.
`tieredcache`	

Package	Purpose
	Hot(memory)/warm(NVMe)/cold(SSD) tiered cache with injected backends + injected clock, idle-demotion + promotion, and per-tier hit stats.
`deltacrdt`	Standalone delta-state CRDTs (G-counter / PN-counter / observed-remove OR-set / LWW-map) with delta-mutators and a convergent (commutative/associative/idempotent) merge.

Scheduling & Placement

Package	Purpose
`scheduler`	Omega-model two-level scheduler: ClassAd matching, gang scheduling (all-or-nothing), preemption, optimistic concurrency.
`constraints`	Pacemaker-style four-type constraint engine (location / colocation / order / stickiness).
`preempt`	Value-multiplier (Gepetto) preemption with sole-global protection.
`priorityqueue`	Multifactor composite priority (age + fairshare + size + QoS) with clock-driven aging.
`backfill`	SLURM-style conservative backfill over an availability timeline.
`workclaim`	SKIP-LOCKED-style optimistic work-claiming (exactly-once under contention).
`admissioncontrol`	N+K capacity-reserve failover admission control.
`budgetcap`	Global MaxCostPerHour budget cap on allocations.
`qos`	QoS-tier routing (real-time / interactive / batch / best-effort).
`suitability`	HPC-vs-inference workload classifier and placement routing.
`ewmarank`	Utilization-aware EWMA candidate ranking with sticky-client routing.
`fallbackchain`	Per-task ordered fallback chains with dedup, attempt cap, and empty-result handling.
`providerchain`	

Package	Purpose
	Ordered multi-tier provider fallback cascade with retriable/terminal error classification and injected-clock failover SLA.
`modelrouter`	Strategy-to-default-model resolution (latency / throughput / quality / cost) with a typed unknown-strategy error.
`gepetto`	HelixGepetto dual-resource local-vs-Chutes arbitration with a monotonic reserve schedule and a strict `>0.80` high-water anti-starvation zero-capacity cut-off.
`smartrouter`	Intelligent ModelRouter (latency / throughput / cost / TEE / balanced) that always excludes unhealthy models, with a typed no-eligible-model error.
`llmfailover`	Error-classified LLM failover taxonomy (deterministic per-class fallback paths) plus a sandbox scheduling-hint contract.

GPU, Resource & Cost Management

Package	Purpose
`pool`	Node/instance pool manager with a `GPUProvider` provisioning seam.
`resources`	Node resource probe (CPU/mem/disk/net/GPU) extended with accelerator reporting: `GPUInfo.TFLOPS` + `Accelerators{NPUTops,FPGALogicElements,QPUPresent}`. Real per-OS probe (darwin Apple-GPU TFLOPS from `system_profiler` core count; linux sysfs PCI-identity catalog) with an operator override-label fallback for non-self-probeable NPU/FPGA/QPU (HXC-1300).
`local`	Local GPU TCO model (amortized capital + power, utilization-scaled).
`costsched`, `latencysched`	Cost-aware and latency-aware schedulers.
`healthmonitor`	Edge-triggered health transitions with auto-failover / recovery callbacks.
`healthprobe`	Startup / readiness / liveness probe tiers with a startup grace period.
`gpuattest`	Device attestation crypto: challenge/response + fingerprint, seeded-matmul proof-of-GPU-work, O(1) Merkle spot-check, device-sealed AES-GCM/HKDF.
`attestadmit`	Attestation-gated scheduler admission predicate (uses `gpuattest`).

Package	Purpose
`quantization`	Per-tier model-variant selection (memory-fit + backend-support gates).
`capability`, `deviceprofile`, `device`, `devicemap`	Capability negotiation and device profiling.
`devicecatalog`	Machine-readable 64-device taxonomy with case-insensitive substring `Lookup` resolving a discovered CPU model (e.g. `Rockchip RK3588`) to its tier / trust-level / compute-class.
`tierdef`, `tiersec`	Tier definitions and tier security.
`burst`, `cloudspot`, `marketplace`	Burst-to-cloud autoscaling, spot pricing, and compute marketplace. `cloudspot` includes real AWS IMDSv2 (token `PUT`→authenticated `GET` instance-action), Azure scheduled-events, and GCP preemption interruption pollers (genuine token-dance wire protocol, `httptest`-proven) driving `drain→checkpoint→upload` via an injectable sink (HXC-1328; live cloud HXC-1617).
`revenueopt`	`RevenueOptimizer.OptimizeAllocation` greedy GPU→marketplace assignment maximising expected revenue, with a TEE→Chutes revenue bias.
`carbonsched`	Carbon-aware job placement: selects the lowest-carbon-intensity latency-eligible region (deterministic name tiebreak) with per-job kWh / gCO2 metering.
`workloadrouter`	`UnifiedManager.RouteWorkload`: concurrent per-marketplace price probes + a weighted composite score (inverse price/latency, health) with a TEE multiplier re-ranking confidential offers when the workload requires a TEE.
`economics`	`RewardDistributor` multi-token distribution with treasury/reinvest splits and a conservation invariant, plus `GetParticipantROI` (ROI% + break-even days, typed unknown-participant error).
`bursthysteresis`	`BurstController` with a MONITOR→SPILL→RECOVER two-threshold hysteresis dead-band.
`gpucatalog`	`SUPPORTED_GPUS` compute-multiplier catalog, `LookupMultiplier`, and attestation-aware `Score` (attested scores rank above un-attested).
`gputopo`	Topology-aware NUMA/NVLink GPU placement scoring that prefers NVLink-connected GPU pairs and falls back to PCIe.
`balancemonitor`	USD account-balance monitor with a strict low-balance floor warning, over HTTP with `Authorization: Bearer`.
`deviceplugin`	Nomad/K8s-style gRPC device-plugin / GRES fingerprinting framework: device plugins register over real gRPC reporting model / memory / driver / PCIe / capabilities / health / utilization; the registry parses GRES descriptors (`gpu:rtx3080:1,memory:10Gi`), atomically applies fingerprints, allocates by request, and rejects oversubscription beyond reported device count.
`gpu`	

Package	Purpose
	`ProviderAdapter` registration hooks: a thread-safe registry (`Register`/`List`/`Get`/`Lookup`, duplicate-name guarded, race-free under `-race`) exposing a pool-facing provider listing so a new external provider adapter plugs into the GPU layer (HXC-1533; pool consumption HXC-1615).
`benchmark`	Micro-benchmarker producing a real, repeatable normalized on-host CPU score (+ typed GPU TFLOPS / NPU TOPS seams) for tier assignment / scheduling hints; repeatability asserted within a coefficient-of-variation tolerance band (HXC-1308).
`tierdetect`	Pre-provision host-capability gate: real cross-platform detection (darwin `sysctl`/arch + linux `/dev/kvm`,`/proc`,`binfmt` behind one build-tagged interface) returning a typed `MissingCapabilityError` when a requested tier's needs are unmet (HXC-1230).

Federation & Multi-cluster

Package	Purpose
`federation` (+ `federation/suspicion`)	Federation membership and suspicion.
`fedtopology`	Federation topology patterns.
`fedtrust`	Cross-cluster federated-trust admission.
`configsync`	CRDT (LWW) config synchronisation.
`residency`	Data-residency admission evaluation.
`raftprofile`	etcd Raft WAN tuning profiles.
`spiffefed`, `doublecrypt`	SPIFFE federation (mTLS + x509) and double-encryption.
`internal/federation`	Karmada PropagationPolicy + OverridePolicy engine: CRs modeled as Go structs → YAML (no client-go dep), constraint-aware (data-locality/latency/cost/compliance) two-level cell selection, and failover reselect that excludes a downed/NotReady cell; deterministic failover proven via in-process FakeHub (live <60s Karmada capture tracked as HXC-924).
`gitops`	ArgoCD ApplicationSet federation client over an injectable HTTP API: matrix generator (one Application per cell), `syncPolicy` prune+selfHeal, RollingSync canary→tier-2 (50%)→tier-1 (25%) progressive rollout, and drift/prune detection; proven against an httptest ArgoCD (live UI capture tracked as HXC-922).

Messaging, Flow & Routing

Package	Purpose
`flowcontrol`	Kubernetes API Priority & Fairness (FlowSchema → PriorityLevel → fair queues).
`workqueue`	Rate-limited work queue with exponential backoff and dedup.
`ratelimit`, `backoff`, `retry`	Rate limiting, exponential backoff, retry.
`idempotent`	Exactly-once producer via producer-id + sequence.
`rebalance`	Kafka cooperative-sticky incremental partition rebalancing.
`informer`	Informer-style list-watch local cache with indexers and resync.
`redundantexec`	BOINC-style redundant-execution trust scorer.
`hashslot`	CRC16 16,384 hash-slot router with MOVED/ASK in-flight migration redirection.
`slotmigration`	Atomic live-session slot migration FSM (PREPARE → TRANSFER → COMMIT/ABORT).
`fiber`, `pubsub`, `events`, `serde`	Framed transport, pub/sub, events, serialization.

Edge

Package	Purpose
`edge`	Edge restriction, schedule rules, trust, and work units.
`edgeregistry`	Multi-tier (T3–T8) edge device registration with tier/trust/capability labels.
`edgeverify`	Redundant edge-output verification against a trusted anchor via SHA-256 checksum.
`edgefusion`	Edge data fusion.
`edgeheartbeat`	Edge heartbeat carrying battery%/charging/CPU-temp/network-type with a churn-tolerant collector that marks a device offline past a configurable timeout; real per-OS `TelemetrySource` (darwin `pmset`/`sysctl`, linux `/sys`+`/proc/net`)

Package	Purpose
	behind one interface, injected clock, validated over a real loopback transport (HXC-1185).
`powergater`	Charging-gated work-acceptance guard: `CanAcceptWork() (bool, reason)` returning `not_charging`/`battery_low_X%`/`daytime`/`cpu_hot_XC` from a real per-OS `PowerSource` (darwin `pmset`, linux `/sys/class/power_supply`+`thermal`) behind one interface (HXC-1175).

Security & Verification

Package	Purpose
`crypto`, `jwt`	Cryptographic primitives and JWT auth.
`modelintegrity`	HF-cache verification gate (SHA-256 + size).
`gravaladmit`	HMAC-SHA256 GraVal challenge-response provider admission gate with single-use nonce and a `graval.verified` label.
`gravalverify`	GraVal GPU attestation with a VRAM-ratio `>= 0.95` gate and a concurrent, race-clean `BatchVerify` pass-rate KPI.
`fsresidency`	Filesystem residency challenge: per-byte-range `ReadAt` + SHA-256 verification proving a model file is really present on disk; truncated / missing / mismatch all fail.
`exportcontrol`	Export-control / country-code KYC gate on node onboarding: a controlled GPU (`H100`/`A100`) from a Tier-3 (embargoed) or unknown jurisdiction is rejected with `ErrExportDenied`; Tier-1 is allowed.
`hybridkex`	Hybrid post-quantum key exchange combining X25519 (`crypto/ecdh`) + ML-KEM-768 (`crypto/mlkem`) shared secrets via HKDF-SHA256 (negotiated group `X25519MLKEM768`); both parties derive an identical 32-byte key, and a tampered ML-KEM ciphertext yields a non-matching key.
`e2eebench`	`MeasureHandshakeLatency` benchmarks the `hybridkex` full handshake (median + P95 over many

Package	Purpose
	iterations), proving median < 1ms on host and enforcing per-iteration key agreement (`ErrKeyDisagreement`).
`compliancedoc`	EU AI Act compliance-documentation pipeline: generates a model card + technical-documentation artifact from attestation-log records with a SHA-256 provenance hash bound to the record content; empty logs → `ErrNoAttestations`.
`gpuattest` (`multigpu.go`)	Multi-GPU node enumeration: an N-GPU node descriptor yields N distinct, independently-verifiable per-device attestation proofs keyed by GPU UUID; rejects duplicate/empty descriptors.
`redundantexec`, `attestadmit`, `gpuattest`	See above — redundancy trust, attestation admission, attestation crypto.

Testing, Simulation & Quality

Package	Purpose
`testing/dst` (+ BUGGIFY, `chaos`, `turmoil`)	FoundationDB-style deterministic simulation testing, BUGGIFY fault macros, network simulation.
`dst`	Standalone FoundationDB-style deterministic simulation harness: single-threaded seeded-RNG event loop with injectable network (drop/delay/reorder) / disk (fault/stall) / logical clock; same seed yields a byte-for-byte identical trace and exact failure replay (independent of `testing/dst`).
`porcupine`	Self-contained WGL linearizability checker + concurrent history recorder: `CheckOperations(model, history)` returns linearizable PASS / FAIL with the violating operation; verifies distributed-op histories under fault injection.
`timefault`	Clock-fault injectors (skew / freeze / monotonic-violation) + skew detector for split-brain avoidance.
`chaosexp`	Chaos experiment suite with delivery-level steady-state.

Package	Purpose
`fmea`	FMEA catalogue + RPN validator.
`phasegate`, `phase7matrix`	Phase dependency gates and the Phase-7 gap-matrix verifier.
`qualitygate`, `covgate`, `stats`, `sandbox`	Quality gates, coverage gates, statistics (Welch t-test), sandboxing.

External Provider Integration

Package	Purpose
`chutes`	Chutes inference client + attestation + E2EE envelope/stream, plus `ChutesMinerConfig` / `ValidatorConfig` deployment descriptors with `Validate()` invariants (non-empty IDs/hotkeys, `GPUCount>0`, non-negative cost, TEE consistency).
`provider/chutes`	OpenAI-compatible `/v1/chat/completions` provider (configurable base URL + `cpk_` Bearer key, `ChatCompletionRequest/Response/Message/Usage`) with HTTP-429 retry + ctx-interruptible backoff that honors `Retry-After`, and typed non-retryable errors.
`provider/runpod`	RunPodProvider serverless GPU adapter (implements `pool.GPUProvider`) with a warm-pool fast path (pre-warmed instances served before cold provisioning) over an injectable client. `Provision` releases the mutex across the `ColdProvision` RPC (reserve-slot → unlock → RPC → relock → record/rollback) so concurrent cold provisions overlap while a `reserved` counter keeps capacity correct; `EndpointOf(id)` exposes the reachable worker endpoint (HXC-919).
`provider/aws`	AWSProvider EC2 Spot adapter (implements `pool.GPUProvider`) over an injectable EC2-client interface (no aws-sdk dependency): GPU model→instance-type selection (p5/p4de/g5), Spot + tagging, and a TOCTOU-safe capacity gate (`launched+reserved`).
`provider/ionet`	IONetProvider over the io.net REST API (`cloud.io.net/api/v1`,

Package	Purpose
	injectable base URL + `*http.Client` + bearer; implements `pool.GPUProvider`): `DeployCluster` (gpu_type/count/region/image/command/env) returns the backend-minted cluster run-UUID (parsed, never fabricated), `HealthCheck` derives healthy state on the active cluster, and a concurrency-safe reserve-under-lock capacity gate; closure proven against a `net/http/httptest` io.net backend.
`chutesaccount`	Chutes API client over HTTP (`Authorization: Bearer`): model-list (TEE / price fields) and `/users/me` account-balance retrieval.
`llmadapter`	Claude Messages / OpenAI Responses-API request and response shape adapters to OpenAI chat-completions, preserving tool-call order.
`marketplaceadapter`	`MarketplaceAdapter` interface (`Name`/`GetCurrentPricing`/`SubmitWork`) with a `Name()`-keyed dispatch `Registry`, an HTTP-backed `ChutesAdapter` (non-2xx rejected), and an `AkashAdapter` (Cosmos/AKT reverse-auction pricing, SDL submission, provider-reputation gate) over an injectable client; routes work to the named marketplace. `buildSDL` YAML-double-quote-encodes caller-controlled `WorkSpec` fields (`yamlQuote`) so a crafted value cannot inject manifest structure, and the reputation gate clamps a non-positive `MinReputation` to a 0.5 safe floor so a zero-value `AkashAdapter{}` cannot admit everyone (HXC-918).

Observability

Package	Purpose
`metrics`	Metrics collection (Prometheus-text `NodeCollector`); plus `TierMetrics` (`tiermetrics.go`) exposing `gpu_tier_utilization`, `gpu_cost_per_hour`, and `provider_health` series, and `EarningsMetrics`

Package	Purpose
	(`earningsmetrics.go`) exposing `tao_earnings_total`, `graval_attestation_status`, `token_throughput`, and `gpu_utilization` series — all with deterministic ordering and concurrent-safe setters.
`tracing`	W3C distributed tracing.
`health`	Health aggregation; includes miner-api (HTTP liveness) and GraVal bootstrap DaemonSet attestation-readiness dependency checks wired into the rollup, which names the specific failing check on a healthy→unhealthy transition.
`grafanadash`	Grafana dashboard generation + validation.
`log`	Structured logging.

This catalogue is maintained as the foundation library grows. For the authoritative list run `ls pkg/`. For the work-item registry that tracks package delivery, see `HXC_REGISTRY.md`.